

Rapport TD4 – SR10

Formulaires POST, Enregistrement de données et Gestion de sessions

Projet *Landr* – Plateforme de recrutement

Mathis Millot
Romain Pierre

Avril 2026

Table des matières

Partie 4 – Formulaires POST et enregistrement des données	2
Configuration d'Express pour les formulaires	2
Inscription en plusieurs étapes	2
Upload de fichiers avec Multer	3
Patron PRG (Post-Redirect-Get)	4
Couche modèle et requêtes SQL paramétrées	4
Partie 5 – Sessions et authentification	5
Configuration des sessions	5
Connexion et création de session	6
Middlewares de contrôle d'accès	6
Déconnexion	7
Authentification Google OAuth	7
Persistance de la session entre les pages	8
Conclusion	8

Partie 4 – Formulaires POST et enregistrement des données

Configuration d'Express pour les formulaires

Pour qu'Express puisse décoder les corps de requête envoyés par les formulaires HTML (`application/x-www-form-urlencoded`), le middleware suivant est enregistré dans `app.js` :

```
1 app.use(express.urlencoded({ extended: true }));
2 app.use(express.json());
```

Sans ce middleware, `req.body` serait `undefined`. L'option `extended: true` permet de parser des objets imbriqués grâce à la bibliothèque `qs`.

Inscription en plusieurs étapes

L'inscription d'un candidat illustre le traitement de formulaires en plusieurs étapes. Les données intermédiaires sont stockées en session entre les requêtes :

```
1 // Etape 1 : POST /inscription_candidat
2 router.post('/inscription_candidat', async (req, res) => {
3   const { nom, prenom, email, num_tel, password } = req.body;
4   req.session.inscription = { nom, prenom, email, num_tel, password };
5   res.redirect('/inscription/etape2');
6 });
7
8 // Etape 2 : POST /inscription/etape2
9 router.post('/inscription/etape2', async (req, res) => {
10   if (!req.session.inscription) return res.redirect('/
11     inscription_candidat');
12   const data = { ...req.session.inscription, ...req.body };
13   const hash = await bcrypt.hash(data.password, 10);
14   const id = await utilisateur.createCandidat({ ...data, password: hash
15     });
16   req.session.user = { id_user: id, nom: data.nom, prenom: data.prenom,
17     email: data.email, role: 'Candidat', statut: '
18     ACTIF' };
19   delete req.session.inscription;
20   res.redirect('/profil_candidat');
21 });
```

Le mot de passe est haché avec `bcrypt` avant insertion, ce qui évite de stocker des données sensibles en clair.

Upload de fichiers avec Multer

La soumission d'une candidature peut inclure des documents (CV, lettre de motivation). Le middleware **Multer** gère l'upload :

```
1 const storage = multer.diskStorage({
2   destination: (req, file, cb) => cb(null, 'public/uploads/'),
3   filename: (req, file, cb) => {
4     const unique = Date.now() + '-' + Math.round(Math.random() * 1e9);
5     cb(null, unique + path.extname(file.originalname));
6   }
7 });
8 const upload = multer({ storage });
```

```

9
10 router.post('/candidature', isCandidat, upload.array('documents', 5),
11   async (req, res) => {
12     const id_offre = parseInt(req.body.id_offre, 10);
13     if (isNaN(id_offre)) return res.redirect('/offres');
14     const offre = await offreModel.read(id_offre);
15     if (!offre) return res.redirect('/offres');
16
17     const id_cand = await candidature.create(req.session.user.id_user,
18       id_offre);
19     for (const file of req.files) {
20       await documentsCandidature.create(id_cand, file.filename);
21     }
22     res.redirect('/candidature/confirmation');
23   });

```

Chaque fichier reçoit un nom unique (timestamp + entier aléatoire) pour éviter les collisions. La validation de `id_offre` via `parseInt` + `isNaN` protège contre les injections ou données malformées.

Patron PRG (Post-Redirect-Get)

Tous les formulaires appliquent le patron **PRG** : après un POST réussi, le serveur répond avec un 302 Redirect vers une page GET. Cela empêche la resoumission du formulaire lors d'un rafraîchissement navigateur.

```

1 res.redirect('/candidature/confirmation'); // PRG apres POST /
  candidature
2 res.redirect('/profil_candidat');          // PRG apres POST /
  modifier_profil

```

Couche modèle et requêtes SQL paramétrées

Les insertions en base passent systématiquement par des requêtes paramétrées pour prévenir les injections SQL :

```

1 // model/candidature.js
2 async create(id_user, id_offre) {
3   const [result] = await db.query(
4     'INSERT INTO Candidature (id_user, id_offre, date, statut) VALUES
5       (?, ?, NOW(), "EN_ATTENTE")',
6     [id_user, id_offre]
7   );
8   return result.insertId;
9 }

```

Les ? sont remplacés par le driver `mysql2` après échappement automatique des valeurs.

Partie 5 – Sessions et authentification

Configuration des sessions

Les sessions sont gérées par **express-session** avec un store MySQL pour la persistance (les sessions survivent aux redémarrages du serveur) :

```

1 // app.js
2 const session      = require('express-session');
3 const MySQLStore   = require('express-mysql-session')(session);
4 const sessionStore = new MySQLStore({ /* memes config que db */ });
5
6 app.use(session({
7   secret: process.env.SESSION_SECRET,
8   resave: false,
9   saveUninitialized: false,
10  store: sessionStore,
11  cookie: { maxAge: 1000 * 60 * 60 * 24 } // 24h
12 }));

```

`saveUninitialized: false` évite de créer une session vide pour chaque visiteur anonyme.

Connexion et création de session

Lors de la connexion, le contrôleur vérifie les identifiants puis stocke les informations essentielles dans `req.session.user` :

```

1 router.post('/connection', async (req, res) => {
2   const { email, password } = req.body;
3   const user = await utilisateur.findByEmail(email);
4   if (!user || !(await bcrypt.compare(password, user.password)))
5     return res.render('html/connection', { error: 'Identifiants
6       invalides' });
7   if (user.statut === 'INACTIF')
8     return res.render('html/connection', { error: 'Compte desactive' });
9
10  req.session.user = {
11    id_user: user.id_user, nom: user.nom, prenom: user.prenom,
12    email: user.email, role: user.role, statut: user.statut,
13    photo_profil: user.photo_profil
14  };
15  // Redirection selon le role
16  const dest = { Admin: '/admin', Recruteur: '/recruteur', Candidat: '/
17    profil_candidat' };
18  res.redirect(dest[user.role] || '/offres');
19 }));

```

Seuls les champs nécessaires sont copiés en session (pas le hash du mot de passe).

Middlewares de contrôle d'accès

Trois middlewares protègent les routes selon le rôle stocké en session :

```

1 function isAdmin(req, res, next) {
2   if (!req.session.user) return res.redirect('/connection');
3   if (req.session.user.role !== 'Admin') return res.redirect('/offres');
4   next();
5 }
6
7 function isRecruteur(req, res, next) {
8   if (!req.session.user) return res.redirect('/connection');

```

```

9   if (req.session.user.role !== 'Recruteur') return res.redirect('/
    offres');
10  next();
11 }
12
13 function isCandidat(req, res, next) {
14   if (!req.session.user) return res.redirect('/connection');
15   if (req.session.user.role !== 'Candidat') return res.redirect('/offres
    ');
16   next();
17 }

```

Ces middlewares sont appliqués directement sur les routes sensibles :

```

1 router.get('/admin', isAdmin, adminController);
2 router.get('/recruteur', isRecruteur, recruteurController);
3 router.post('/candidature', isCandidat, upload.array('documents', 5)
    , ...);

```

Déconnexion

La déconnexion détruit la session côté serveur et redirige vers la page de connexion :

```

1 router.post('/logout', (req, res) => {
2   req.session.destroy(() => res.redirect('/connection'));
3 });

```

Authentification Google OAuth

En complément de la connexion locale, une authentification via **Google OAuth 2.0** est implémentée avec `passport` et `passport-google-oauth20`. La callback crée ou retrouve l'utilisateur en base (`findOrCreateByGoogle`), puis initialise la même structure de session que la connexion locale, assurant une cohérence complète du système d'autorisation quel que soit le mode d'authentification.

Persistence de la session entre les pages

La session persiste entre toutes les navigations grâce au cookie de session envoyé par le navigateur. Les vues EJS accèdent à `req.session.user` passé via `res.render` :

```

1 // Exemple dans la route GET /profil_candidat
2 router.get('/profil_candidat', isCandidat, async (req, res) => {
3   const candidatures = await candidatureModel.readByCandidat(
4     req.session.user.id_user
5   );
6   res.render('html/profil_candidat', {
7     user: req.session.user, candidatures
8   });
9 });

```

Conclusion

Le projet Landr met en œuvre l'ensemble des concepts abordés dans le TD 4 :

- Traitement de formulaires POST avec `express.urlencoded` et `req.body`
- Validation et sanitisation des données entrantes (`parseInt`, `isNaN`, `bcrypt`)
- Upload de fichiers via Multer avec nommage sécurisé
- Patron PRG pour éviter les doubles soumissions
- Requêtes SQL paramétrées pour prévenir les injections
- Sessions persistantes en MySQL avec `express-session` + `express-mysql-session`
- Contrôle d'accès granulaire par rôle via des middlewares dédiés
- Authentification locale (`bcrypt`) et OAuth (Google)